

Contrato de Endpoints — Módulos Contratos y Pagos/Mora

Versión: 1.0
Fecha: 2026-04-24
Estado: Borrador para revisión del equipo
Frontend: Emily Perea Córdoba
Backend: Por definir

Convenciones generales

Parámetro	Valor
Base URL	NEXT_PUBLIC_API_URL (variable de entorno)
Formato de fecha	ISO 8601: YYYY-MM-DD
Moneda	Pesos colombianos (COP), siempre como number entero
Paginación	page (base 1) y limit en query params; respuesta incluye total y totalPaginas
Errores	Objeto { mensaje: string, codigo?: string } en el campo error
Autenticación	Bearer token en header Authorization (JWT — interceptor pendiente en src/proxy.ts)
Content-Type	application/json salvo endpoints de upload de archivos (multipart/form-data)

Estructura de respuesta estándar

```
// Éxito – recurso único
{ data: T }

// Éxito – listado paginado
{
  data: T[]
  total: number // total de registros que coinciden con el filtro
  pagina: number
  totalPaginas: number
}

// Error
{
  error: {
    mensaje: string // mensaje legible
    codigo?: string // código interno opcional, ej: "CONTRATO_NO_ENCONTRADO"
```

```
}  
}
```

Tipos compartidos

Estos tipos son referenciados en múltiples endpoints. El backend debe respetarlos exactamente.

```
type TipoContrato = "arriendo" | "promesa_compraventa"  
  
type EstadoContrato =  
  | "borrador"  
  | "en_firmas"  
  | "activo"  
  | "en_escrituracion"  
  | "pendiente_registro"  
  | "por_vencer"  
  | "vencido_con_saldos"  
  | "terminacion_en_disputa"  
  | "terminado_anticipadamente"  
  | "finalizado"  
  
type TipoCobro =  
  | "canon"  
  | "comision_administracion"  
  | "comision_colocacion"  
  | "arras"  
  | "deposito"  
  | "penalizacion"  
  | "precio_venta"  
  
type EstadoCobro = "pendiente" | "pagado" | "en_mora"  
  
type TipoEventoCuenta =  
  | "cobro_generado"  
  | "pago_recibido"  
  | "mora_iniciada"  
  | "interes_mora"  
  
type FormaPago = "contado" | "credito_hipotecario" | "mixto"  
  
type TipoDocumento = "CC" | "NIT" | "CE" | "PAS"
```

Módulo: Contratos

1. Listar contratos

GET /contratos

Query params (todos opcionales):

Param	Tipo	Descripción
busqueda	string	Filtra por referencia, inmueble, propietario o contraparte (búsqueda parcial, case-insensitive)
estado	EstadoContrato	Filtra por estado exacto
tipo	TipoContrato	Filtra por tipo de contrato
asesor	string	Nombre parcial del asesor
page	number	Página (default: 1)
limit	number	Resultados por página (default: 20)

Respuesta exitosa 200:

```
{
  data: ContratoResumen[]
  total: number
  pagina: number
  totalPaginas: number
  resumenEstados: Record<EstadoContrato, number> // conteo por estado, para las
tarjetas del encabezado
}

interface ContratoResumen {
  id: string
  referencia: string // ej: "CTR-2025-001"
  tipo: TipoContrato
  estado: EstadoContrato
  inmueble: string // nombre descriptivo del inmueble
  direccion: string
  propietario: string // nombre completo
  contraparte: string // nombre completo
  asesor: string
  fechaInicio: string // ISO date
  fechaFin: string // ISO date
  valorCanon: number // COP
}
```

2. Obtener detalle de contrato

GET /contratos/:id

Respuesta exitosa 200:

```
{
  data: ContratoDetalle
}

interface ContratoDetalle {
  id: string
  referencia: string
  tipo: TipoContrato
  estado: EstadoContrato
  fechaInicio: string
  fechaFin: string
  asesor: string

  inmueble: {
    id: string
    nombre: string
    direccion: string
    ciudad: string
  }

  propietario: {
    id: string
    nombre: string
    documento: string
    tipoDocumento: TipoDocumento
    telefono: string
    email: string
  }

  contraparte: {
    id: string
    nombre: string
    documento: string
    tipoDocumento: TipoDocumento
    telefono: string
    email: string
  }

  codeudor?: {
    nombre: string
    documento: string
  }

  // Solo si tipo === "arriendo"
  condicionesArriendo?: {
    valorCanon: number
  }
}
```

```

    diaCorte: number // 1-31
    incluyeAdministracion: boolean
    valorAdministracion?: number
    tieneDeposito: boolean
    valorDeposito?: number
    duracionMeses: number
}

// Solo si tipo === "promesa_compraventa"
condicionesPromesa?: {
    precioVenta: number
    valorArras: number
    fechaLimiteArras: string
    formaPago: FormaPago
    entidadFinanciera?: string // si formaPago incluye credito
    fechaAprobacionCredito?: string
    valorContado?: number // si formaPago es "mixto"
    valorCredito?: number // si formaPago es "mixto"
    fechaEscrituracion?: string
    notaria?: string
}

documentos: DocumentoContrato[]
firmas: FirmaContrato[]
historial: EventoHistorial[]
}

interface DocumentoContrato {
    id: string
    nombre: string // ej: "Cédula arrendatario"
    tipo: string // clave interna del tipo de doc
    estado: "pendiente" | "recibido" | "rechazado"
    fechaSubida?: string
    urlArchivo?: string
}

interface FirmaContrato {
    id: string
    parte: string // nombre de la persona
    rol: string // ej: "Arrendador", "Arrendatario"
    estado: "pendiente" | "firmado"
    fechaFirma?: string
}

interface EventoHistorial {
    id: string
    tipo: string // ej: "creacion", "firma", "activacion",
"vencimiento"
    descripcion: string
    fecha: string
    usuario?: string // quién realizó la acción
}

```

Errores:

- 404 — Contrato no encontrado

3. Crear contrato de arriendo

POST /contratos/arriendo

Body:

```
{
  inmuebleId: string
  contraparteId: string           // cliente (arrendatario)
  asesor: string

  fechaInicio: string            // ISO date
  duracionMeses: number          // 6-36

  valorCanon: number
  diaCorte: number              // 1-31

  incluyeAdministracion: boolean
  valorAdministracion?: number   // requerido si incluyeAdministracion === true

  tieneDeposito: boolean
  tipoDeposito?: "meses_canon" | "valor_fijo"
  mesesDeposito?: number        // si tipoDeposito === "meses_canon"
  valorDeposito?: number        // si tipoDeposito === "valor_fijo"

  tieneCodeudor: boolean
  codeudor?: {
    nombre: string
    documento: string
  }
}
```

Respuesta exitosa 201:

```
{
  data: {
    id: string
    referencia: string
    estado: "borrador"           // siempre inicia en borrador
  }
}
```

Errores:

- 400 — Datos inválidos (campos faltantes o fuera de rango)
 - 404 — Inmueble o cliente no encontrado
 - 409 — El inmueble ya tiene un contrato activo
-

4. Crear contrato de promesa de compraventa

POST /contratos/promesa

Body:

```
{
  inmuebleId: string
  contraparteId: string          // cliente (comprador)
  asesor: string

  precioVenta: number
  valorArras: number
  fechaLimiteArras: string       // ISO date

  formaPago: FormaPago

  // Requerido si formaPago === "credito_hipotecario" o "mixto"
  entidadFinanciera?: string
  fechaAprobacionCredito?: string

  // Requerido si formaPago === "mixto"
  valorContado?: number
  valorCredito?: number

  fechaEscrituracion?: string
  notaria?: string
}
```

Respuesta exitosa 201:

```
{
  data: {
    id: string
    referencia: string
    estado: "borrador"
  }
}
```

Errores:

- 400 — Datos inválidos
 - 404 — Inmueble o cliente no encontrado
 - 409 — El inmueble ya tiene un contrato activo
-

5. Transición de estado: Renovar contrato

```
POST /contratos/:id/renovar
```

Solo disponible si `estado === "activo"` o `estado === "por_vencer"`.

Body:

```
{
  nuevaFechaFin: string      // ISO date
  nuevoValorCanon?: number   // si se ajusta el valor
}
```

Respuesta exitosa 200:

```
{
  data: {
    id: string
    estado: EstadoContrato // nuevo estado después de renovar
    fechaFin: string
    valorCanon: number
  }
}
```

Errores:

- 400 — Estado del contrato no permite renovación
 - 404 — Contrato no encontrado
-

6. Transición de estado: Iniciar terminación

```
POST /contratos/:id/terminar
```

Solo disponible si `estado === "activo"` o `estado === "por_vencer"`.

Body:

```
{
  motivo: string
  fechaTerminacion: string // ISO date
  enDisputa: boolean // si true → estado pasa a
  "terminacion_en_disputa"
}
```

Respuesta exitosa 200:

```
{
  data: {
    id: string
    estado: "terminado_anticipadamente" | "terminacion_en_disputa"
  }
}
```

Errores:

- 400 — Estado del contrato no permite terminación
- 404 — Contrato no encontrado

7a. Actualizar firmas del contrato

```
PATCH /contratos/:id/firmas
```

Permite registrar que una o más partes han firmado el contrato. Pasa el estado de "borrador" a "en_firmas" si no estaba en ese estado.

Body:

```
{
  firmas: {
    parte: string // nombre de la persona
    rol: string // ej: "Arrendador", "Arrendatario"
    estado: "firmado" // solo aceptan "firmado" en este endpoint
  }[]
}
```

Respuesta exitosa 200:

```
{
  data: {
    id: string
    estado: EstadoContrato // generalmente pasa a "en_firmas" o avanza
según lógica
    firmas: FirmaContrato[]
  }
}
```

Errores:

- 404 — Contrato no encontrado
- 400 — Contrato no está en estado "borrador" o "en_firmas"

7b. Registrar escrituración (promesas)

POST /contratos/:id/escriturar

Solo disponible si `tipo === "promesa_compraventa"` y `estado === "en_escrituracion"`.

Body:

```
{
  fechaEscrituracion: string
  notaria: string
  numeroEscritura?: string
}
```

Respuesta exitosa 200:

```
{
  data: {
    id: string
    estado: "pendiente_registro"
  }
}
```

8. Gestionar documentos del contrato

8a. Listar documentos

```
GET /contratos/:id/documentos
```

Respuesta 200:

```
{
  data: DocumentoContrato[]
}
```

8b. Subir documento

```
POST /contratos/:id/documentos
Content-Type: multipart/form-data
```

Form fields:

Campo	Tipo	Descripción
tipo	string	Tipo de documento (ej: "cedula_arrendatario")
nombre	string	Nombre descriptivo
archivo	File	PDF o imagen

Respuesta 201:

```
{
  data: DocumentoContrato
}
```

8c. Actualizar estado de documento

```
PATCH /contratos/:id/documentos/:docId
```

Body:

```
{
  estado: "recibido" | "rechazado"
  nota?: string // motivo de rechazo
}
```

Respuesta 200:

```
{
  data: DocumentoContrato
}
```

8d. Eliminar documento

```
DELETE /contratos/:id/documentos/:docId
```

Respuesta 204: Sin cuerpo.

8e. Descargar documento

```
GET /contratos/:id/documentos/:docId/descargar
```

Respuesta 200: Binary file (PDF o imagen) con header `Content-Disposition: attachment; filename="nombre.pdf"`.

Errores:

- 404 — Documento no encontrado o sin archivo adjunto

9. Eliminar contrato

```
DELETE /contratos/:id
```

Solo disponible si el contrato está en estado "borrador". Sirve para limpiar contratos incompletos sin eliminar referencias en historiales.

Respuesta exitosa 204: Sin cuerpo.

Errores:

- 404 — Contrato no encontrado
- 400 — El contrato no está en estado "borrador"

Módulo: Cobros y Pagos

10. Listar cobros de un contrato

```
GET /contratos/:id/cobros
```

Query params (todos opcionales):

Param	Tipo	Descripción
estado	EstadoCobro	Filtrar por estado
tipo	TipoCobro	Filtrar por tipo de cobro
page	number	Página (default: 1)
limit	number	Resultados por página (default: 50)

Respuesta exitosa 200:

```
{
  data: Cobro[]
  resumen: {
    totalPendiente: number
    totalEnMora: number
    totalPagado: number
  }
  pagina: number
  totalPaginas: number
}

interface Cobro {
  id: string
  tipo: TipoCobro
  estado: EstadoCobro
  periodo?: string // ej: "Marzo 2025" – solo aplica a cobros recurrentes
  fechaLimite: string // fecha máxima de pago
  valor: number // COP
  diasMora?: number // solo si estado === "en_mora"
  interesesMora?: number // COP, solo si aplica (inmuebles comerciales)
  esInmuebleResidencial: boolean // determina si aplica ley 820 (sin intereses)
  comprobante?: {
    url: string
    fechaPago: string
  }
}
```

11. Obtener detalle de un cobro

```
GET /cobros/:cobroId
```

Respuesta exitosa 200:

```
{
  data: CobroDetalle
}

interface CobroDetalle extends Cobro {
  contrato: {
    id: string
    referencia: string
    tipo: TipoContrato
  }
  inmueble: {
    nombre: string
    direccion: string
  }
  cliente: {
    id: string
    nombre: string
    email: string
    telefono: string
  }
  historialPagos: PagoRegistrado[]
}

interface PagoRegistrado {
  id: string
  fecha: string
  valor: number
  notas?: string
  comprobanteUrl?: string
  registradoPor?: string
}
```

12. Registrar pago de un cobro

POST /cobros/:cobroId/pagos
Content-Type: multipart/form-data

Form fields:

Campo	Tipo	Descripción
fecha	string	Fecha del pago (ISO date)
comprobante	File	PDF del comprobante (requerido)

Campo	Tipo	Descripción
notas	string	Notas opcionales

Respuesta exitosa **201**:

```
{
  data: {
    pagoId: string
    cobroId: string
    nuevoEstadoCobro: EstadoCobro // generalmente "pagado"
    comprobante: {
      url: string
      fechaPago: string
    }
  }
}
```

Errores:

- **400** — Fecha o comprobante faltantes
- **404** — Cobro no encontrado
- **409** — El cobro ya está pagado

13. Estado de cuenta de un contrato (timeline)

```
GET /contratos/:id/estado-cuenta
```

Query params (todos opcionales):

Param	Tipo	Descripción
desde	string	ISO date — filtrar eventos desde esta fecha
hasta	string	ISO date — filtrar eventos hasta esta fecha

Respuesta exitosa **200**:

```
{
  data: EventoCuenta[]
  saldoActual: number // saldo total pendiente al momento
}

interface EventoCuenta {
  id: string
  tipo: TipoEventoCuenta
}
```

```
fecha: string // ISO datetime
descripcion: string // ej: "Canon mensual – Marzo 2025"
monto: number // positivo = cargo, negativo = abono
saldoAcumulado: number // saldo después de este evento
notas?: string // ej: "Pago tardío – 12 días de mora"
cobroId?: string // referencia al cobro relacionado
pagoId?: string // referencia al pago relacionado
}
```

Módulo: Mora

14. Cobros en mora (dashboard global)

```
GET /cobros/mora
```

Query params (todos opcionales):

Param	Tipo	Descripción
contratoId	string	Filtrar por contrato específico
page	number	Página (default: 1)
limit	number	Default: 50

Respuesta exitosa 200:

```
{
  data: ContratoEnMora[]
  resumen: {
    totalEnMora: number // suma total de valores en mora (COP)
    interesesAcumulados: number // suma de intereses (COP)
    contratosAfectados: number // número de contratos distintos con mora
  }
  pagina: number
  totalPaginas: number
}

interface ContratoEnMora {
  contrato: {
    id: string
    referencia: string
    tipo: TipoContrato
  }
  inmueble: {
    nombre: string
    direccion: string
  }
}
```

```

    esResidencial: boolean // determina si aplica ley 820
  }
  cliente: {
    id: string
    nombre: string
  }
  cobrosEnMora: CobroMora[]
  urgencia: "alta" | "media" | "baja" // alta: 30+ días, media: 15-29, baja:
<15
}

interface CobroMora {
  id: string
  tipo: TipoCobro
  periodo?: string
  fechaLimite: string
  diasMora: number
  valor: number // valor original del cobro
  interesesMora: number // COP - 0 si esResidencial
}

```

Módulo: Reportes de pagos

15. Reporte de pagos recibidos

GET /reportes/pagos

Query params (todos opcionales):

Param	Tipo	Descripción
desde	string	ISO date
hasta	string	ISO date
clienteId	string	ID del cliente
inmuebleId	string	ID del inmueble
tipo	TipoCobro	Tipo de cobro
page	number	Default: 1
limit	number	Default: 50

Respuesta exitosa 200:

```

{
  data: PagoReporte[]
}

```

```

    resumen: {
      totalRecibido: number // suma de todos los pagos filtrados (COP)
    }
    pagina: number
    totalPaginas: number
  }

  interface PagoReporte {
    pagoId: string
    fecha: string // fecha en que se registró el pago
    contrato: {
      id: string
      referencia: string
      tipo: TipoContrato
    }
    inmueble: {
      nombre: string
      direccion: string
    }
    cliente: {
      nombre: string
    }
    tipoCobro: TipoCobro
    periodo?: string // ej: "Marzo 2025"
    valor: number // COP
  }

```

Endpoints relacionados requeridos por el frontend

Estos endpoints son necesarios para formularios dentro de contratos y pagos, aunque pertenecen a otros módulos.

16. Listar todos los inmuebles

```
GET /inmuebles
```

Query params (todos opcionales):

Param	Tipo	Descripción
disponibles	boolean	Si true, solo inmuebles sin contrato activo (default: false)
busqueda	string	Filtrar por nombre, dirección (búsqueda parcial)
tipo	string	"casa" "apartamento" "local" "otro"
page	number	Página (default: 1)
limit	number	Resultados por página (default: 50)

Respuesta 200:

```

{
  data: InmuebleOpc[]
  pagina?: number // opcional si limit no se usa
  totalPaginas?: number
}

interface InmuebleOpc {
  id: string
  nombre: string // ej: "Apto 301 Torre A"
  direccion: string
  ciudad: string
  tipo: "casa" | "apartamento" | "local" | "otro"
  modalidad: "arriendo" | "venta" | "ambos"
  propietario: {
    id: string
    nombre: string
  }
}

```

17. Clientes (para selector en crear contrato)

```
GET /clientes?busqueda=:texto
```

Query params:

Param	Tipo	Descripción
busqueda	string	Nombre o documento (mínimo 2 caracteres)
tipo	string	"arrendatario" "comprador" (filtra por tipo de cliente)

Respuesta 200:

```

{
  data: ClienteOpc[]
}

interface ClienteOpc {
  id: string
  nombre: string
  documento: string
  tipoDocumento: TipoDocumento
  tipos: string[]
  email: string
  telefono: string
}

```

Resumen de endpoints

#	Método	Ruta	Descripción
1	GET	/contratos	Listar contratos con filtros
2	GET	/contratos/:id	Detalle de contrato
3	POST	/contratos/arriendo	Crear arriendo
4	POST	/contratos/promesa	Crear promesa de compraventa
5	POST	/contratos/:id/renovar	Renovar contrato
6	POST	/contratos/:id/terminar	Iniciar terminación
7a	PATCH	/contratos/:id/firmas	Actualizar firmas
7b	POST	/contratos/:id/escriturar	Registrar escrituración
8a	GET	/contratos/:id/documentos	Listar documentos
8b	POST	/contratos/:id/documentos	Subir documento
8c	PATCH	/contratos/:id/documentos/:docId	Actualizar estado de doc
8d	DELETE	/contratos/:id/documentos/:docId	Eliminar documento
8e	GET	/contratos/:id/documentos/:docId/descargar	Descargar documento
9	DELETE	/contratos/:id	Eliminar contrato (solo borrador)
10	GET	/contratos/:id/cobros	Cobros de un contrato (paginado)
11	GET	/cobros/:cobroId	Detalle de cobro
12	POST	/cobros/:cobroId/pagos	Registrar pago
13	GET	/contratos/:id/estado-cuenta	Timeline estado de cuenta
14	GET	/cobros/mora	Dashboard de mora (paginado)
15	GET	/reportes/pagos	Reporte de pagos (paginado)
16	GET	/inmuebles	Listar todos los inmuebles (paginado)
17	GET	/clientes?busqueda=	Cientes para selector

Notas de negocio para el backend

- Generación automática de cobros:** Al crear un contrato de arriendo, el backend debe generar automáticamente los cobros iniciales: depósito (si aplica) y comisión de colocación. Los cobros de canon se generan cada mes en el **diaCorte**.

2. **Ley 820 — intereses de mora:** Los inmuebles residenciales NO pueden cobrar intereses de mora. Solo aplica a locales comerciales. El campo `esResidencial` en `Cobro` y `CobroMora` permite que el frontend lo muestre correctamente.
3. **Pagos parciales:** No soportados en esta versión. Un cobro queda en `"en_mora"` hasta recibir el 100% del valor. El endpoint de registrar pago debe validar que el monto sea igual al valor del cobro.
4. **Transiciones de estado de contrato:** El backend debe validar las transiciones permitidas. Un `"borrador"` no puede pasar directo a `"activo"` sin pasar por `"en_firmas"`.
5. **valorCanon en listado:** El campo `valorCanon` en `ContratoResumen` para promesas de compraventa debe contener el `precioVenta`.
6. **Referencia de contrato:** El campo `referencia` (ej: `"CTR-2025-001"`) lo genera el backend automáticamente. El frontend no lo envía al crear.
7. **Mora automática:** El backend es responsable de cambiar el estado de un cobro de `"pendiente"` a `"en_mora"` cuando se supera la `fechaLimite`. El frontend solo lee el estado calculado.